

Лекция 5

Темы

- 1) Использование web-разметки.
- 2) Протокол https.
- 3) Разработка безопасных web-приложений (концептуальный подход, основы)

Рассматриваемые темы, организационные вопросы

Разметка HTML документа....

Должна соблюдаться

простота кода, понятность кода, читабельность кода

Разметка HTML-документа (Простота)

Простая

```
<form class="modal-form" action="this" method="post">
  <input type="text" placeholder="Логин">
  <input type="password" placeholder="Пароль">
  <div class="clearfix row">
    <div class="save-me to-left">
      <input type="checkbox" id="remember-me">
      <label for="remember-me">Запомните меня</label>
    </div>
    <div class="new-password to-right">
      <a href="#">Я забыл пароль!</a>
    </div>
  </div>
  <button type="submit" class="btn">Войти</button>
</form>
```

```
<form class="login-form" action="/echo" method="post">
  <input type="text" placeholder="Логин">
  <input type="password" placeholder="Пароль">
  <a href="#" class="restore">Я забыл пароль!</a>
  <input type="checkbox" name="remember" id="remember-me">
  <label for="remember-me">Запомните меня</label>
  <button type="submit" class="btn">Войти</button>
</form>
```

Разметка HTML-документа (Понятность)

Понятная

Следует обратить особое внимание на правильное именование классов

.cart

.main-menu

.catalog-item

.nbr-razdela

.siniy-treugolnik

.idishnik

Класс не описательный, а для чего он предназначен

Разметка HTML-документа (Понятность)

Что это за блок или блоки?

```
<div class="slider-content">
  
</div>
<div class="button-combination clearfix">
  <a href="#" class="btn to-left">...</a>
  <a href="#" class="btn to-right">...</a>
</div>
```

```
<div class="gallery">
  <figure class="gallery-content">
    
  </figure>
  <button class="btn gallery-prev" disabled>...</button>
  <button class="btn gallery-next">...</button>
</div>
```

Разметка HTML-документа (Читабельность кода)

Форматирование кода за счёт отступов и переносов в HTML и CSS.

Какой режим работы?

```
<div class="main-contacts"><h2
class="content-column-title">Контактная
    информация</h2><p>
    Barbershop «Borodinski»<br>Адрес: г. Санкт-Петербург, ул. Собаки
    Павлова, д. 13<br>Телефон: +7 (495) 666-02-66
</p><p>Время работы:<br>пн – пт: с 10:00
    до 22:00<br>сб – вс: с 10:00 до 19:00</p><a
        href="#" class="btn">Как проехать</a>
<a href="#" class="btn">Обратная связь</a></div>
```

```
<div class="main-contacts">
  <h2 class="content-column-title">Контактная информация</h2>
  <p>
    Barbershop «Borodinski»<br>
    Адрес: г. Санкт-Петербург, ул. Собаки Павлова, д. 13<br>
    Телефон: +7 (495) 666-02-66
  </p>
  <p>
    Время работы:<br>
    пн – пт: с 10:00 до 22:00<br>
    сб – вс: с 10:00 до 19:00
  </p>
  <a href="#" class="btn">Как проехать</a>
  <a href="#" class="btn">Обратная связь</a>
</div>
```


Разметка HTML-документа (Читабельность кода)

Корректное использование пространства имен

HTML:

```
<div class="features-item">  
  <div class="icon"></div>  
  <h2 class="title">Практические занятия</h2>  
  <p>На одной теории далеко не уедешь, поэтому мы обучаем на практике.</p>  
</div>
```

CSS:

```
.features-item {...}  
.features-item .icon {...}  
.features-item .title {...}  
.features-item p {...}
```

Каждый элемент, который нужно стилизовать, предваряем с помощью контекстного или ложного селектора классом родительского элемента. Получается, что никакой из элементов, находящихся внутри блока, не выпадает в глобальную область видимости.

Разметка HTML-документа (Читабельность кода)

Корректное использование пространства имен

HTML:

```
<div class="features-item">  
  <div class="features-icon"></div>  
  <h2 class="features-title">Практические занятия</h2>  
  <p>На одной теории далеко не уедешь, поэтому мы обучаем на практике.</p>  
</div>
```

CSS:

```
.features-item {...}  
.features-icon {...}  
.features-title {...}  
.features-item p {...}
```

В каждое имя класса вставляем префикс родителя, в разметке у нас получаются достаточно длинные имена классов, но в CSS красивый плоский список селекторов. По префиксу класса можно понять, к какому именно блоку этот класс относится.

пространства имён

- Упрощают именование классов.
- Предотвращают нежелательное перемешивание стилей.
- Разбивают код на независимые блоки.
- Ускоряют разработку и упрощают поддержку.

Теги article, section и div.

<ARTICLE>, <SECTION>, <DIV>

- **ARTICLE** — самостоятельный, независимый, отчуждаемый «кусоч» содержимого.
- **SECTION** — составная часть чего-либо, может именоваться.
- **DIV** — чисто декоративное назначение, никакой семантики.

Подходы к стилю кодирования

Стиль кодирования — дело личных предпочтений или соглашений внутри команды.

Главное, чтобы стиль кодирования применялся:

- последовательно.
- единообразно.

Стайлгайд от MDO

<http://mdo.github.io/code-guide/>

Перевод: <http://instanceof.pro/code-guide/>

Стайлгайд от GOOGLE

[http://google-](http://google-styleguide.googlecode.com/svn/trunk/htmlcssguide.xml)

[styleguide.googlecode.com/svn/trunk/htmlcssguide.xml](http://google-styleguide.googlecode.com/svn/trunk/htmlcssguide.xml)

Idiomatic CSS

<http://github.com/necolas/idiomatic-css/>

Тестирование...

Опрос 4

🔖 [Добавить страницу в мои закладки](#)

Опрос 4

0/1 point (ungraded)

Зачем нужен макет-стайлгайд?

- ☐ В нём содержится набор направляющих для построения сетки страницы;
- ☐ В нём нарисованы различные состояния управляющих элементов (кнопок и ссылок при наведении, полей форм в состоянии фокуса и так далее);
- ☒ В нём содержатся правила использования тех или иных графических элементов сайта на разных носителях: на конвертах, визитках, бланках и так далее. ❌

Опрос 1

🔖 [Добавить страницу в мои закладки](#)

Опрос 1

2/2 points (ungraded)

1. С помощью какого тега вы бы разметили пост в соцсети?

☒ < article > ✓

☐ < section >

☐ < div >

2. Какой стиль кодирования лучше?

☐ Разделитель классов — дефис, отступы задаются пробелами, размер отступа — 2 пробела.

☐ Разделитель классов — нижнее подчёркивание, отступы задаются пробелами, размер отступа — 4 пробела.

☒ Ни один из стилей кодирования не лучше другого, главное, чтобы они применялись единообразно и последовательно. ✓

Поток документа, блочная модель документа

Посмотреть самостоятельно. Вопросы будут в тесте. Если надо, то можем разобрать подробнее на следующей лекции. Используйте форум курса <https://moodle.itmo.ru/course/view.php?id=33> для того, чтобы задавать вопросы и давать обратную связь...

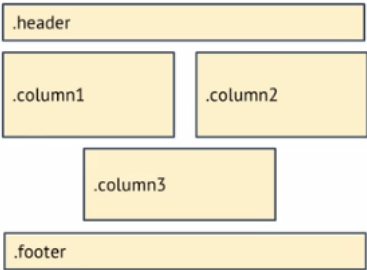
Поток документа, блочная модель документа

Добавить страницу в мои закладки

Поток документа, блочная модель документа

Что такое поток документа?

```
<body>
  <div class="header"></div>
  <div class="column1"></div>
  <div class="column2"></div>
  <div class="column3"></div>
  <div class="footer"></div>
</body>
```



В нем есть пять элементов: хедер, три колонки и футер.

По умолчанию браузер отображает элементы слева направо и сверху вниз, поэтому эти пять дивов отобразятся как пять прямоугольников, которые расположены друг под другом в том порядке, в котором они расположены в коде.

Однако этот поток документа можно изменить и сделать, например, таким: первая и вторая колонки расположены в колонке, третья расположена под ними и отцентрирована.

Тот же самый кусок html-кода мы можем изменить, чтобы он отображался в другом потоке, то есть хедер сверху, потом три колонки и внизу футер.

Что же такое "сетка"?

Для этого давайте посмотрим на наш учебный макет.

Вот он перед вами, это главная страница нашей школы, которую мы будем верстать.

И сеткой обычно называют относительное расположение крупных блоков.

Блоки обычно выделяют сверху вниз: самые крупные и потом блоки внутри них.

Если посмотреть на нашу сетку, то отчетливо

Видео

Скачать видео

Субтитры

Скачать SubRip (.srt) file

Скачать Text (.txt) file

1) Использование web-разметки.

2) Протокол https.

3) Разработка безопасных web-приложений (концептуальный подход, основы)

https 443

HTTPS (аббр. от [англ. HyperText Transfer Protocol Secure](#)) — расширение [протокола HTTP](#) для поддержки [шифрования](#) в целях повышения безопасности.

Данные в протоколе HTTPS передаются поверх криптографических протоколов [TLS](#) или устаревшего в 2015 году [SSL](#). В отличие от HTTP с [TCP-портом](#) 80, для HTTPS по умолчанию используется TCP-порт 443.

Протокол был разработан компанией [Netscape Communications](#) для браузера [Netscape Navigator](#) в 1994 году.

HTTPS	
Название	HyperText Transfer Protocol Secure
Уровень (по модели OSI)	Прикладной
Семейство	TCP/IP
Создан в	2000
Порт/ID	443/TCP
Назначение протокола	Безопасное соединение с сервером
Спецификация	RFC 2818 
Основные реализации (клиенты)	веб-браузеры
Основные реализации (серверы)	Apache , nginx , IIS

https

HTTPS не является отдельным протоколом. Это обычный HTTP, работающий через шифрованные транспортные механизмы [SSL](#) и [TLS](#)^[4]. Он обеспечивает защиту от атак, основанных на прослушивании сетевого соединения — от [снифферских](#) атак и атак типа [man-in-the-middle](#), при условии, что будут использоваться шифрующие средства и *сертификат сервера проверен и ему доверяют*.

По умолчанию HTTPS URL использует 443 [TCP-порт](#) (для незащищённого HTTP — 80). Чтобы подготовить веб-сервер для обработки https-соединений, администратор должен получить и установить в систему сертификат открытого и закрытого ключа для этого веб-сервера. В TLS используется как [асимметричная схема шифрования](#) (для выработки общего секретного ключа), так и [симметричная](#) (для обмена данными, зашифрованными общим ключом). Сертификат открытого ключа подтверждает принадлежность данного открытого ключа владельцу сайта. Сертификат открытого ключа и сам открытый ключ посылаются клиенту при установлении соединения; закрытый ключ используется для расшифровки сообщений от клиента.

Существует возможность создать такой сертификат, не обращаясь в [центр сертификации](#). Подписываются такие сертификаты этим же сертификатом и называются [самоподписанными](#) (*self-signed*). Без проверки сертификата каким-то другим способом (например, звонок владельцу и проверка контрольной суммы сертификата) такое использование HTTPS подвержено [атаке посредника](#).

https

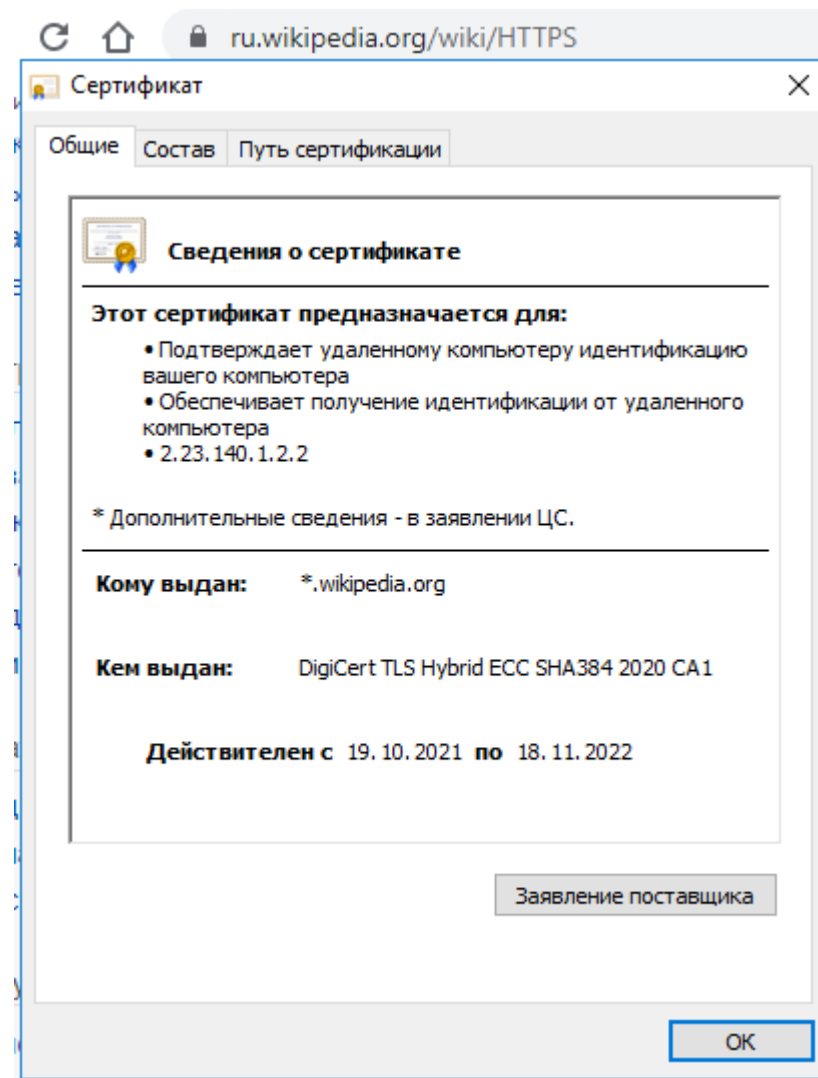
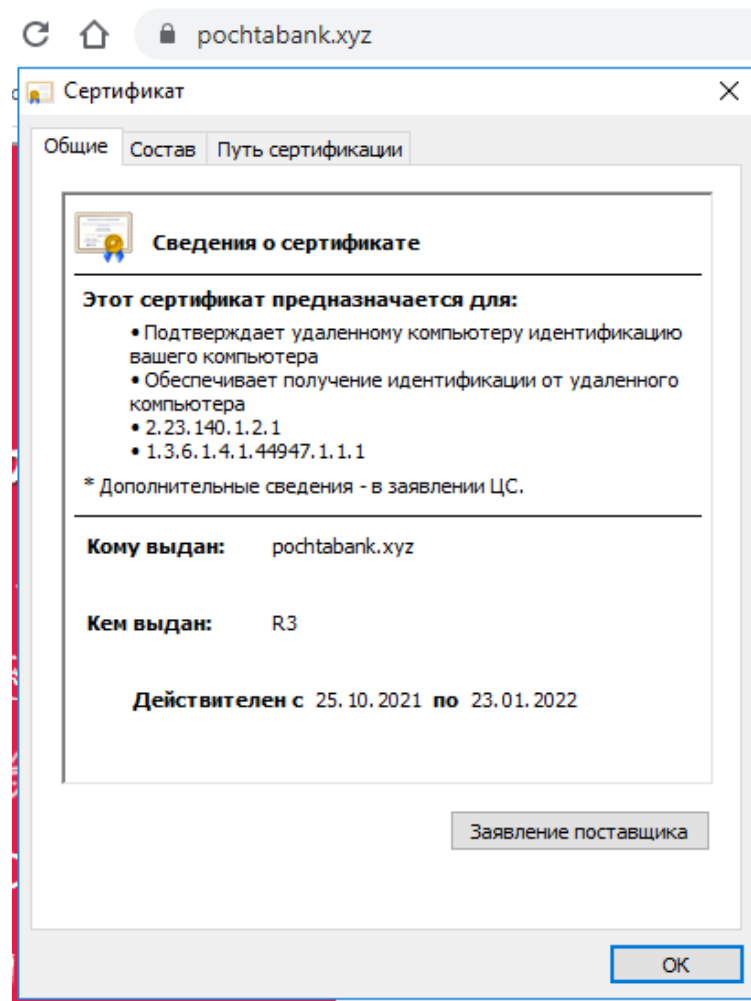
Эта система также может использоваться для аутентификации клиента, чтобы обеспечить доступ к серверу только [авторизованным пользователям](#). Для этого администратор обычно создаёт сертификаты для каждого пользователя и загружает их в браузер каждого пользователя. Также будут приниматься все сертификаты, подписанные организациями, которым доверяет сервер. Такой сертификат обычно содержит имя и адрес электронной почты авторизованного пользователя, которые проверяются при каждом соединении, чтобы проверить личность пользователя без ввода пароля.

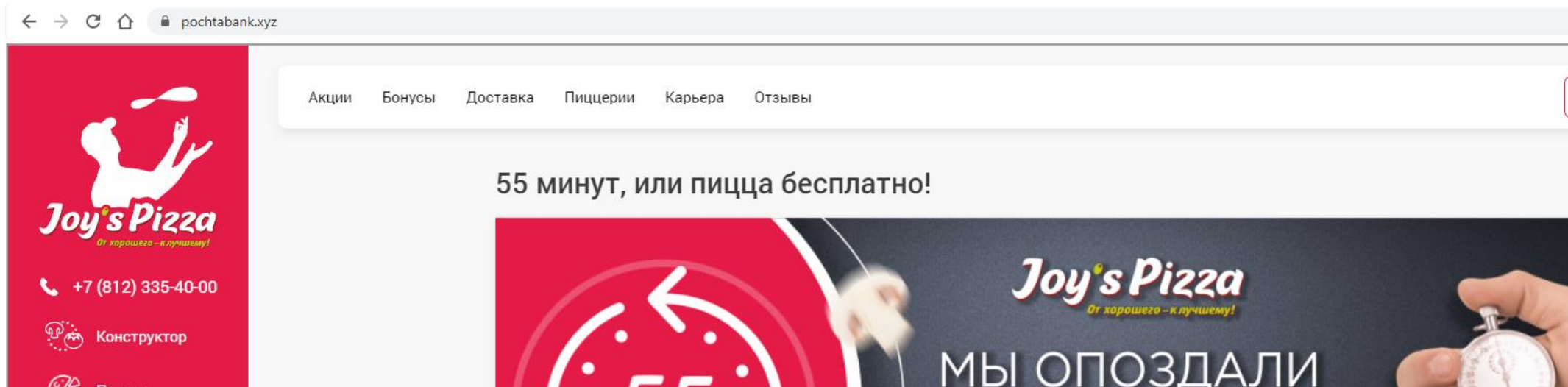
В HTTPS для шифрования используется длина ключа 40, 56, 128 или 256 бит. Некоторые старые версии браузеров используют длину ключа 40 бит (пример тому — [IE](#) версий до 4.0), что связано с экспортными ограничениями в США. Длина ключа 40 бит не является надёжной. Многие современные сайты требуют использования новых версий браузеров, поддерживающих шифрование с длиной ключа 128 бит, с целью обеспечить достаточный уровень безопасности. Шифрование с длиной ключа 128 бит значительно затрудняет подбор паролей и доступ к личной информации.

Традиционно на одном IP-адресе может работать только один HTTPS-сайт. Для работы нескольких HTTPS-сайтов с различными сертификатами применяется расширение TLS под названием [Server Name Indication](#) (SNI).

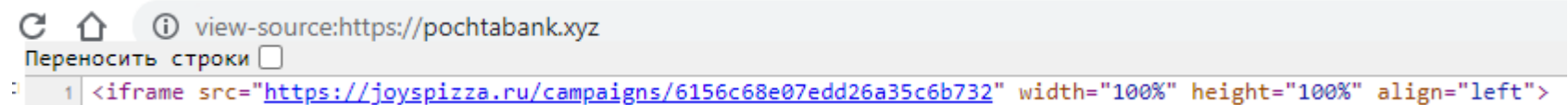
https

Демонстрация (смотрим примеры сертификатов)





<iframe src="<https://joyspizza.ru/campaigns/6156c68e07edd26a35c6b732>" width="100%" height="100%" align="left">



1) Использование web-разметки.

2) Протокол https.

**3) Разработка безопасных web-приложений
(концептуальный подход, основы)**

 **Безопасность веб-приложений. Разведка, защита, нападение. [2021] Эндрю Хоффман**

Эта книга не просто поможет защитить ваше веб-приложение от хакеров, но также расскажет об их тактике поиска уязвимостей и взлома. Мы обсудим множество приемов, которыми пользуются современные хакеры для взлома веб-приложений, поддерживаемых корпорациями, правительствами, а иногда и любителями. После тщательного изучения этих методов мы перейдем к средствам защиты. Вы сможете по-новому взглянуть на архитектуру приложений, а также научитесь применять передовой опыт обеспечения безопасности при их разработке.



**Безопасность веб-приложений.
Разведка, защита, нападение. [2021]
Эндрю Хоффман**

Эта книга не просто поможет защитить ваше веб-приложение от хакеров, но также расскажет об их тактике поиска уязвимостей и взлома. Мы обсудим множество приемов, которыми пользуются современные хакеры для взлома веб-приложений, поддерживаемых корпорациями, правительствами, а иногда и любителями. После тщательного изучения этих методов мы перейдем к средствам защиты. Вы сможете по-новому взглянуть на архитектуру приложений, а также научитесь применять передовой опыт обеспечения безопасности при их разработке.

Эта книга не просто поможет защитить ваше веб-приложение от хакеров, но также расскажет об их тактике поиска уязвимостей и взлома. Мы обсудим множество приемов, которыми пользуются современные хакеры для взлома веб-приложений, поддерживаемых корпорациями, правительствами, а иногда и любителями. После тщательного изучения этих методов мы перейдем к средствам защиты. Вы сможете по-новому взглянуть на архитектуру приложений, а также научитесь применять передовой опыт обеспечения безопасности при их разработке.

 [Скачать книгу](#)

https://t.me/hack_theory/433

Лучшей практикой часто считается предоставление сведений об ошибках в более общей форме.

Современные браузеры предлагают надежную изоляцию для страниц разных сайтов в соответствии с правилом ограничения домена, которое еще называют принципом одинакового источника (**Same Origin Policy, SOP**). Согласно этому принципу, веб-сайт А не может быть доступен сайту В, даже если оба открыты в браузере одновременно или один встроен в другой с помощью тега `iframe`.

Появилась у браузеров и новая настройка безопасности, известная как Политика безопасного контента (**Content Security Policy, CSP**). Она позволяет разработчику сайта выбирать уровень безопасности: например, указывать, нужно ли выполнять встроенные функции `inline` (в HTML). Это дает дополнительную защиту приложения от распространенных угроз.

Часто современные веб-приложения состоят из нескольких, связанных через так называемый REST API. REST расшифровывается как **Representational State Transfer** («передача состояния представления»). Такой API не сохраняет состояния и существует только для выполнения запросов одного приложения к другому. Это означает, что он не хранит сведения об инициаторе запроса.

В архитектуре REST каждый запрос считается независимой транзакцией, то есть общение клиента с сервером состоит из независимых пар «запрос–ответ». При этом на сервере не сохраняется никакой информации о состоянии клиента. Это не означает, что REST API не может выполнять аутентификацию и авторизацию. Просто вместе с каждым запросом отправляется информация, идентифицирующая пользователя, например, в виде токена.

Большинство современных REST API передают данные в формате JSON.

JSON, открытый (некоммерческий) текстовый формат обмена данными, имеющий ряд интересных характеристик:

- лаконичный (не требует большой пропускной способности сети);
- легко интерпретируется (это снижает нагрузку на серверное/клиентское оборудование);
- легко читается человеком;
- допускает иерархическое представление данных (что позволяет выражать сложные соотношения);
- представления объектов JSON и объектов JavaScript очень похожи, что упрощает создание новых объектов JSON в браузере.

В браузере JSON может быть легко преобразован в объект JavaScript

Для анализа сетевого трафика в браузере Chrome нужно сделать следующее:

1. Щелкните по иконке с тремя точками в правом верхнем углу панели навигации, чтобы открыть меню.
2. Наведите указатель мыши на пункт **Дополнительные инструменты** и выберите вариант **Инструменты разработчика (Developer tools)**.
3. В верхнем меню открывшегося окна перейдите на вкладку **Network**. Если этой вкладки не видно, разверните окно по горизонтали.

The screenshot shows a Chrome browser window with the address bar displaying 'mctrewards.ru'. The page content includes the site name 'MCTREWARDS.RU' and a list of links. The DevTools Network tab is open, showing a list of network requests with columns for Name, Status, Type, Initiator, Size, Time, and Waterfall. The 'Network' tab is highlighted in the top bar of DevTools.

Name	Status	Type	Initiator	Size	Time	Waterfall
www.mctrewards.ru	200	document	Other	104 kB	470 ms	
style.min.css?ver=5.8.1	200	stylesheet	(index)	10.7 kB	70 ms	
css?family=Playfair+Display%3A400%2C700%2C900%7...	200	stylesheet	(index)	634 B	86 ms	
magnific-popup.min.css?ver=5.8.1	200	stylesheet	(index)	1.8 kB	68 ms	
blocks.min.css?ver=5.8.1	200	stylesheet	(index)	1.8 kB	68 ms	
slick.min.css?ver=5.8.1	200	stylesheet	(index)	671 B	68 ms	
slick-theme.min.css?ver=5.8.1	200	stylesheet	(index)	912 B	83 ms	
style.css?ver=5.8.1	200	stylesheet	(index)	25.4 kB	85 ms	
blue.min.css?ver=5.8.1	200	stylesheet	(index)	1.0 kB	83 ms	
style.css?ver=1.3.6	200	stylesheet	(index)	418 B	314 ms	
jquery.min.js?ver=3.6.0	200	script	(index)	31.1 kB	269 ms	

Итоги лекции

Вопросы ?